

BUENAS PRACTICAS PARA APLICACIONES REACTIVAS

D. F. Velásquez Pichilla

7690-16-3882 Universidad Mariano Gálvez

Seminario de Tecnología de Información

dvelasquezp4@miumg.edu.gt

URL REPOSITORIO LA TEX:

<https://github.com/DiegoVelasq98/foro-buenas-practicas-apps-reactivas.git>

Resumen

En este artículo se desarrolló un análisis sobre las buenas prácticas para el desarrollo de aplicaciones reactivas. Como punto de partida, se abordó la definición de este paradigma de programación, fundamentado en la propagación eficiente de cambios, el manejo de eventos de forma asíncrona y el uso de flujos de datos observables que permiten responder de manera dinámica a los cambios del entorno. Asimismo, se describieron los principios que caracterizan a las aplicaciones reactivas, entre ellos la capacidad de respuesta, la resiliencia, la elasticidad y el intercambio de mensajes asíncronos, elementos que favorecen un bajo acoplamiento entre los componentes y una mejor gestión de errores y recursos. De igual forma, se analizaron los beneficios de aplicar buenas prácticas durante el desarrollo, destacando la organización del código, la automatización de procesos, la facilidad de mantenimiento, la reducción de errores y la optimización del rendimiento. Como resultado, se concluye que la correcta implementación de estas prácticas permite construir aplicaciones capaces de adaptarse dinámicamente a diferentes cargas de trabajo, responder oportunamente a eventos concurrentes y ofrecer una experiencia confiable, estable y eficiente para el usuario final, incluso en escenarios de alta demanda o ante la presencia de fallos parciales.

Palabras clave: asincrono, modulos, reactivo, escalabilidad, eficiencia.

Desarrollo del tema

Las aplicaciones reactivas son aquellas que nacen del paradigma de la programación asíncrona. Siendo así que tiende a enfocarse en manejar el flujo de los datos para luego reaccionar a los cambios en tiempo real. Todo esto naciendo debido al gran auge que tenían las aplicaciones web, en las cuales su objetivo es el brindar una buena experiencia a los usuarios, además de la capacidad de respuestas a dichas acciones. Ya que la forma tradicional en que se abordaba estas necesidades, se quedaban cortas en la complejidad de las solicitudes y al crecimiento en cuanto a la demanda de los servicios. Siendo la solución a dichos problemas las aplicaciones reactivas, las cuales ya abordaban el crear aplicaciones con mejores capacidades para responder y que a su vez no afecten la interacción del usuario, como lo podría ser el consumir más recursos. Las aplicaciones reactivas se basan principalmente en cuatro pilares fundamentales que se encargan de que estas respondan, sean escalables y robustas:

- **Capacidad de respuesta:** Esto indica que los tiempos de respuesta se deben garantizar que sean lo más rápido posible. Ya que esto va a permitir brindar el servicio aunque las cargas sean demasiado pesadas o sucedan posibles errores de sistema.
- **Disponibilidad:** Esto indica que siempre deben de tratar de estar funcionando aunque lleguen a ocurrir errores o caídas, deben de ser capaces de poder recuperarse rápidamente y seguir operando con normalidad. Siendo dicho de otra forma que si ocurre un error, sea posible aislarlo y que no comprometa todo el funcionamiento de la aplicación, siendo más fácil de detectar el error ya aislado y darle solución.

- **Adaptabilidad:** Esto indica que siempre deben de poder adaptarse a las cargas que presenten de forma automatica, siendo que si es bajo , debe de destinar los recursos minimos, pero si es un flujo alto, destinar mas recursos segun sea necesario. Para poder garantizar el procesar las solicitudes sin afectar el rendimiento ni tiempos de respuesta.
- **Arquitectura basada en mensajes:** Esto indica que la comunicacion entre los componentes debe ser independiente, utilizando el intercambio de mensajes de forma asincrona para evitar la dependencia, lo que permite una buena escalabilidad y flexibilidad del sistema.

Ventajas de aplicar buenas practicas

Aporta varios beneficios al desarrollo moderno. Ya que ayuda a los desarrolladores a crear aplicaciones mas eficientes, escalables y a la vez que se minimizan los errores y el mantenimiento a las mismas. Entre estas se pueden mencionar:

- **Rendimiento:** Ayudan a que el funcionamiento de las aplicaciones sea mas rapido debido a que los bloques independientes no deben de esperar y quedarse bloqueados. Ya que en lugar de eso, todos funcionan al mismo tiempo y dar respuesta simplemente cuando ocurra un evento. Siendo asi que todo el procesamiento es mas rapido y permite el ahorro de recursos del sistema.
- **Uso de recursos:** Debido a que se basa principalmente en asincronia, suelen consumir menos recursos. Siendo de gran ayuda cuando se maneja un numero alto de usuarios concurrentes.
- **Tiempos de respuesta en UI/UX:** Debido a su enfoque reactivo, hace que cada vez que se realizar un nuevo evento, todo se actualice correctamente en la pantalla, sin que el usuario tenga que recargar nuevamente, todo es en tiempo real en base a eventos.
- **Errores simplificados:** Cualquier error que se presente se maneja de forma centralizada, lo que permite un mejor control y aislamiento del mismo, sin tener que comprometer todo el funcionamiento de la aplicacion, garantizando que aun siga en pie.
- **Escalabilidad:** Debido a su enfoque que es capaz de soportar multiples cargas de trabajo, garantiza que siga teniendo un buen rendimiento sin importar que el numero de usuarios o complejidad del sistema aumente mas.

Bibliotecas y herramientas

A medida que aumenta mas las aplicaciones reactivas, aumentan la cantidad de herramientas que hacen posible su desarrollo:

- **RxJava:** Permite el uso de secuencias observables para manejar eventos asincronos, concurrencia y errores de forma eficiente.
- **Proyector Reactor:** Es una libreria reactiva que permite manejar el flujo de datos de tipo asincronos, facilitando el desarrollo de apps concurrentes y eficientes evitando la programacion no bloqueante.
- **AKKA:** Se basa en el modelo Actor para construir sistemas concurrentes y tolerantes a fallos, facilitando la comunicacion entre componentes distribuidos.
- **RSocket:** Permite la comunicacion reactiva eficiente entre cliente y servidor, manejando la contrapresion.

Buenas practicas

- **Adoptar la inmutabilidad:** Esto ayuda a evitar problemas al no modificar directamente sobre los datos. Esto nos garantiza el flujo coherente y predecible de los datos a lo largo de la aplicacion.
- **Adoptar tecnologias y herramientas estandarizadas:** Es importante seleccionar el grupo de herramientas que se adapten al tipo de necesidad de sistema reactivo a implementar, para aprovechar sus beneficios, como la contrapersion,escalabilidad, etc.
- **Flujos de eventos:** Siempre se debe tener con anticipacion como sera el flujo de cada evento en el sistema, tales como la gestion de errores, terminacion, gestion de procesos, con el fin de poder tener un mejor analisis y desarrollo.
- **Eficiencia en los recursos:** Es vital el definir cada uno de los procesos para minimizar las latencias y costos en comunicacion, mas si se trabajan en entornos de la nube.
- **Buen manejo de componentes:** El trabajar con modulos ayuda a simplificar el proceso y a permitir que sea posible su escalabilidad si es necesario. Ademas de permitir que todo actue de forma independiente , facilitando el mantenimiento.
- **Centrarse en el diseño sin estado:** Ayuda a mejorar el crecimiento y el control de los errores para prevenir que cada uno de los componentes dependan de datos persistentes. Minimiza la atencion de eventos y mejora el mantenimiento.
- **Elegir siempre la asincronia:** Se usa con el fin de lograr tener mas eficiencia y recepcion de eventos. Este enfoque no genera bloqueos y mejora el uso de los recursos, ademas de manejar los eventos para buscar el maximo rendimiento y la posibilidad de ser escalable a futuro.
- **Manejo de contrapresion:** Es vital para que los usuarios regulen el ritmo de los datos, para evitar las sobrecargas y que el flujo del proceso sea eficiente incluso si hay mucha demanda del sistema.

Observaciones y comentarios

Aunque las aplicacion reactivas dan muchos beneficios, su implementacion requiere de un conocimiento profundo desde las bases y herramientas. Ademas es necesario considerar los desafios que puede haber en cada necesidad en especifico, siendo asi una oportunidad de estudio para futuros trabajos de investigacion.

Las aplicaciones reactivas tambien representan un enorme desafio ya que se deben de tener muy definido en si como es que se comportan los temas de asincronia y eventos, siendo que para un desarrollador que empieza desde 0 no seria recomendable debido a la enorme curva de aprendizaje que tiene y que aun crece día a día.

Conclusiones

Se llego a la conclusion de que las aplicaciones reactivas si se siguen correctamente las recomendaciones de su implementacion, se obtienen enormes resultados que benefician tanto al cliente como al sistema.

Se llego a la conclusion de que las aplicaciones reactivas son capaces de no hacer que todo el funcionamiento este disponible aun si se encuentra con fallas, ya que debido a su sistema modular

e independiente, es mas sencillo encontrar el error, sin comprometer la disponibilidad completa de la aplicacion.

Egrafía

Outsystems. (s.f.). Best practices for reactive web apps.

<https://www.outsystems.com/blog/posts/best-practices-reactive-web-app/>

Manoj, M. (2022). Reactive programming: A beginner's guide. freeCodeCamp.

<https://www.freecodecamp.org/news/reactive-programming-beginner-guide/>

AppMaster. (2023). Programación reactiva: Arquitectura de software moderna.

<https://appmaster.io/es/blog/programacion-reactiva-arquitectura-de-software-moderna>

Rouse, M. (s.f.). Reactive programming. TechTarget.

<https://www.techtarget.com/searchapparchitecture/definition/reactive-programming>

Garrido, A., Carrillo, J. (2013). Programación reactiva en la administración de proyectos: aproximación conceptual y aplicaciones prácticas. Revista EAN, (74), 72-85.